

Assignment No: 09

1. Title

Creating an Application in Salesforce using Apex Programming Language

2. Problem Statement

Salesforce is a cloud-based CRM platform that allows developers to build custom applications using Apex programming language.

The objective of this assignment is to create an application in Salesforce using Apex and manage data through custom objects and logic.

3. Objectives

- To understand the basics of Salesforce platform
- To learn Apex programming
- To create custom objects and fields
- To develop logic using Apex classes
- To perform CRUD operations

4. Introduction

4.1 What is Salesforce?

Salesforce is a cloud-based Customer Relationship Management (CRM) platform used for managing business data and applications.

4.2 What is Apex?

Apex is an object-oriented programming language used in Salesforce to execute flow and transaction control statements on the platform.

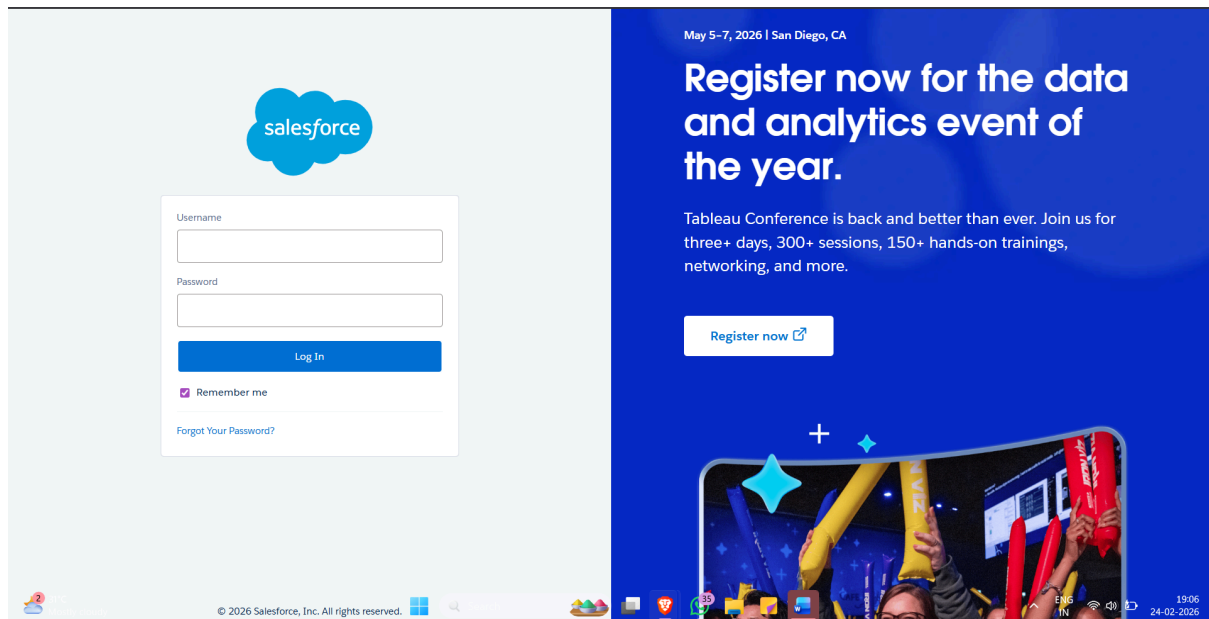
5. Requirements

- Salesforce Developer Account
- Internet Connection
- Basic knowledge of programming

6. Steps to Perform: Creating Application in Salesforce

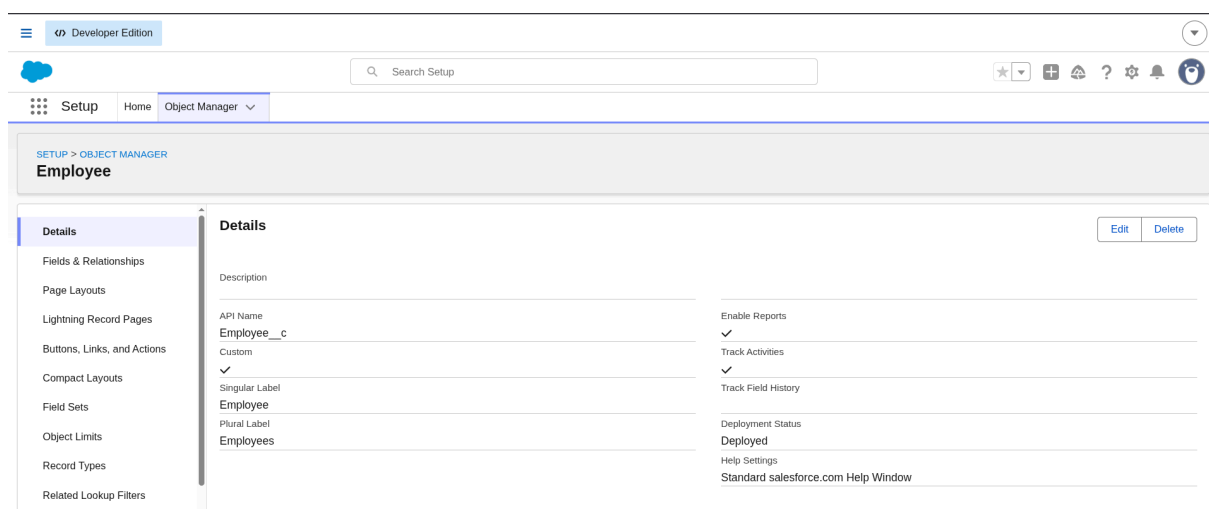
1. Account Access

- **Sign up & Login:** Go to developer.salesforce.com/signup to create your free Developer Edition. Once your email is verified, log in at login.salesforce.com.



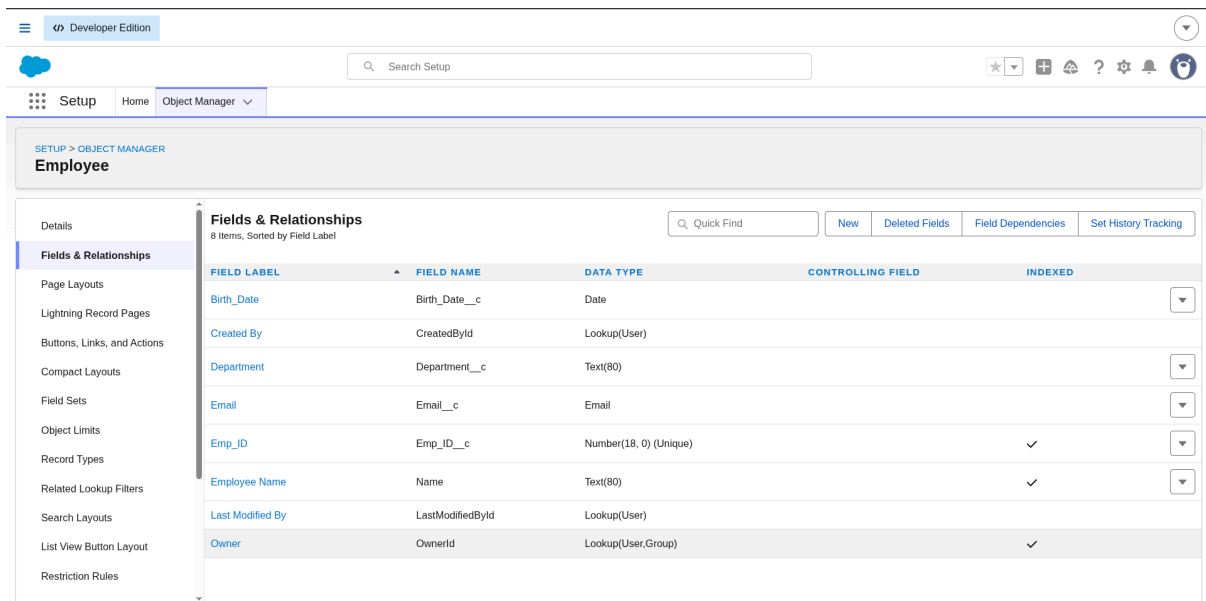
Step: Create Custom Object

Go to Setup → Object Manager → Click Create → Custom Object → Enter Label: Employee, Object Name: Employee → Click Save



Step 3 : Create Fields in Object

- Open Employee Object
- Click **Fields & Relationships** → **New**
- Add fields:
 - Name (Text)
 - Email (Email)
 - Department (Text)
 - Salary (Number)

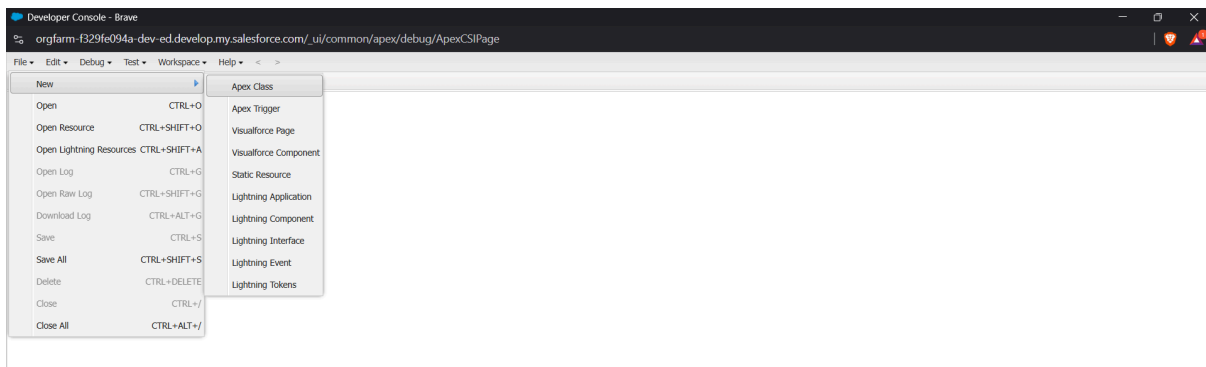


The screenshot shows the Salesforce Setup interface for the Employee object. The left sidebar contains a navigation menu with options like Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Search Layouts, List View Button Layout, and Restriction Rules. The main content area is titled 'Employee' and 'Fields & Relationships', showing 8 items sorted by Field Label. A table lists the fields with columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed status.

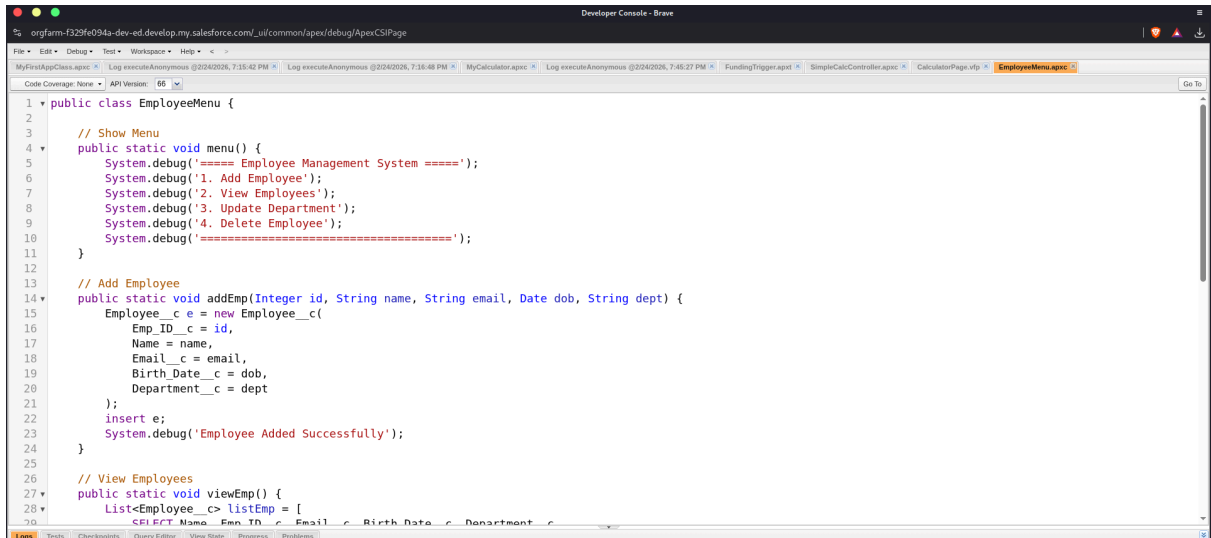
FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Birth Date	Birth_Date__c	Date		
Created By	CreatedById	Lookup(User)		
Department	Department__c	Text(80)		
Email	Email__c	Email		
Emp ID	Emp_ID__c	Number(18, 0) (Unique)		✓
Employee Name	Name	Text(80)		✓
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓

Step 4: Create Apex Class

- **Create New File:** In the Developer Console, go to **File > New > Apex Class**.



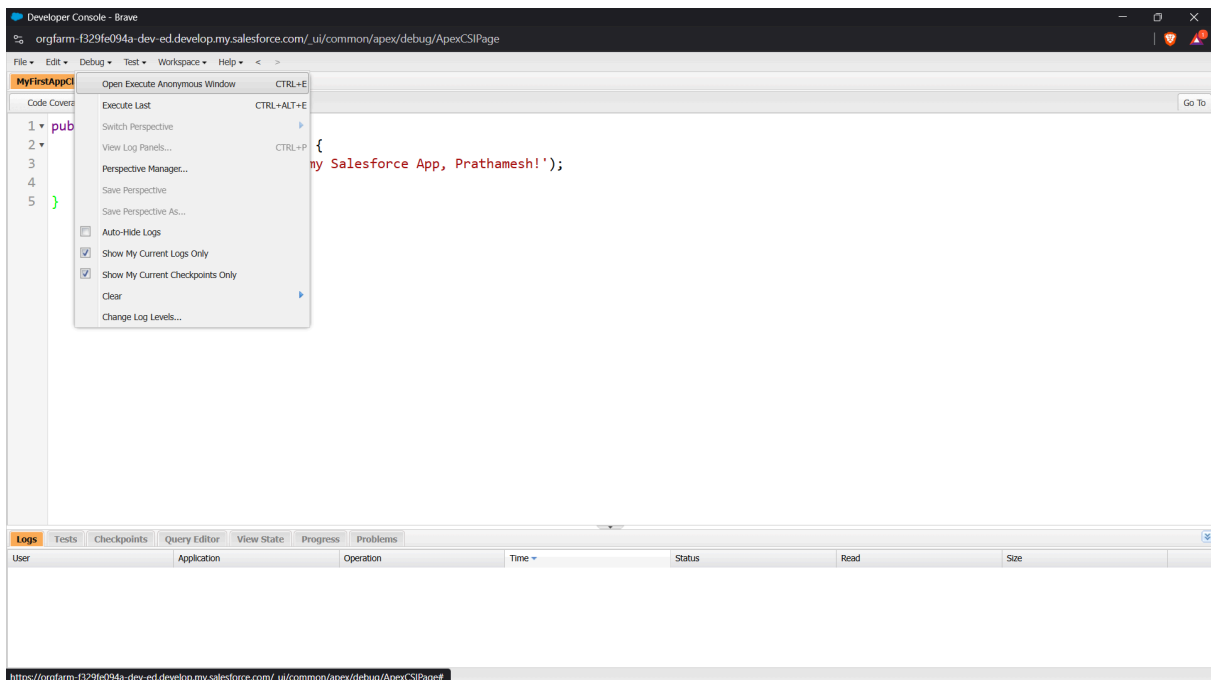
The screenshot shows the Salesforce Developer Console interface. The 'File' menu is open, displaying a list of options including New, Open, Open Resource, Open Lightning Resources, Open Log, Open Raw Log, Download Log, Save, Save All, Delete, Close, and Close All. The 'New' option is selected, and a sub-menu is visible showing 'Apex Class' as the first option, followed by Apex Trigger, Visualforce Page, Visualforce Component, Static Resource, Lightning Application, Lightning Component, Lightning Interface, Lightning Event, and Lightning Tokens.



```
1 public class EmployeeMenu {
2
3     // Show Menu
4     public static void menu() {
5         System.debug('===== Employee Management System =====');
6         System.debug('1. Add Employee');
7         System.debug('2. View Employees');
8         System.debug('3. Update Department');
9         System.debug('4. Delete Employee');
10        System.debug('=====');
11    }
12
13    // Add Employee
14    public static void addEmp(Integer id, String name, String email, Date dob, String dept) {
15        Employee__c e = new Employee__c(
16            Emp_ID__c = id,
17            Name = name,
18            Email__c = email,
19            Birth_Date__c = dob,
20            Department__c = dept
21        );
22        insert e;
23        System.debug('Employee Added Successfully');
24    }
25
26    // View Employees
27    public static void viewEmp() {
28        List<Employee__c> listEmp = [
29            SELECT Name, Emp_ID__c, Email__c, Birth_Date__c, Department__c
30            FROM Employee__c
31        ];
32    }
33 }
```

Step 6: Execute Apex Code

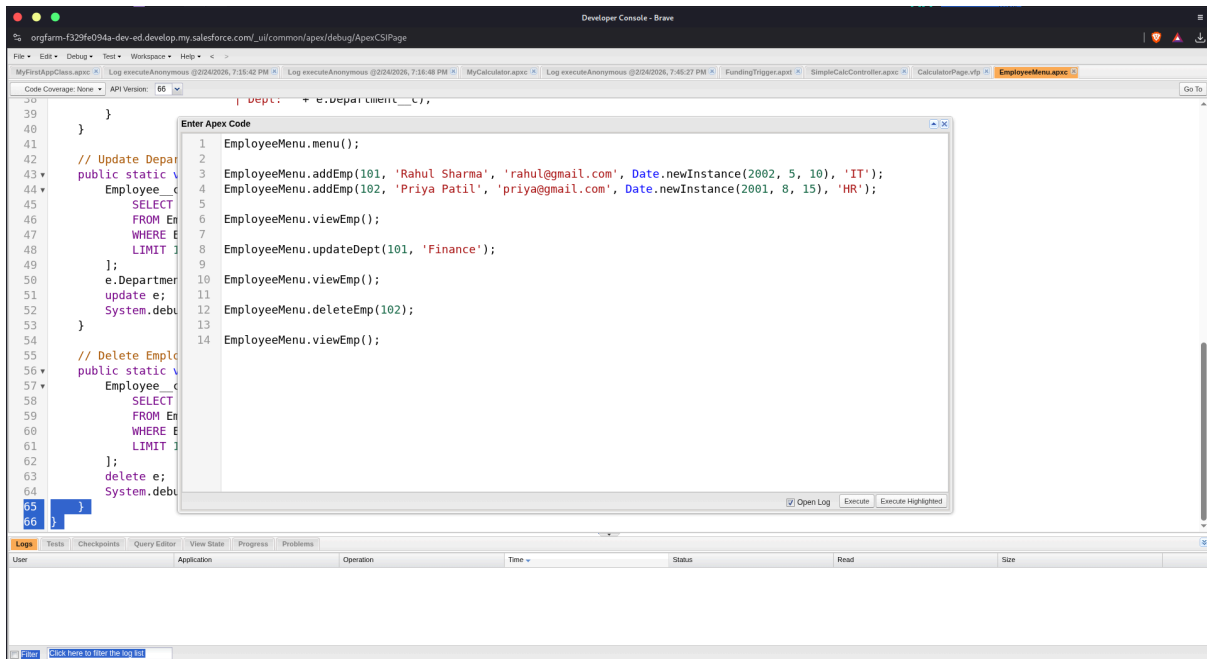
- Go to **Developer Console**
- Click **Debug** → **Open Execute Anonymous Window**



- **Execute Anonymous:** Click on **Open Execute Anonymous Window**.

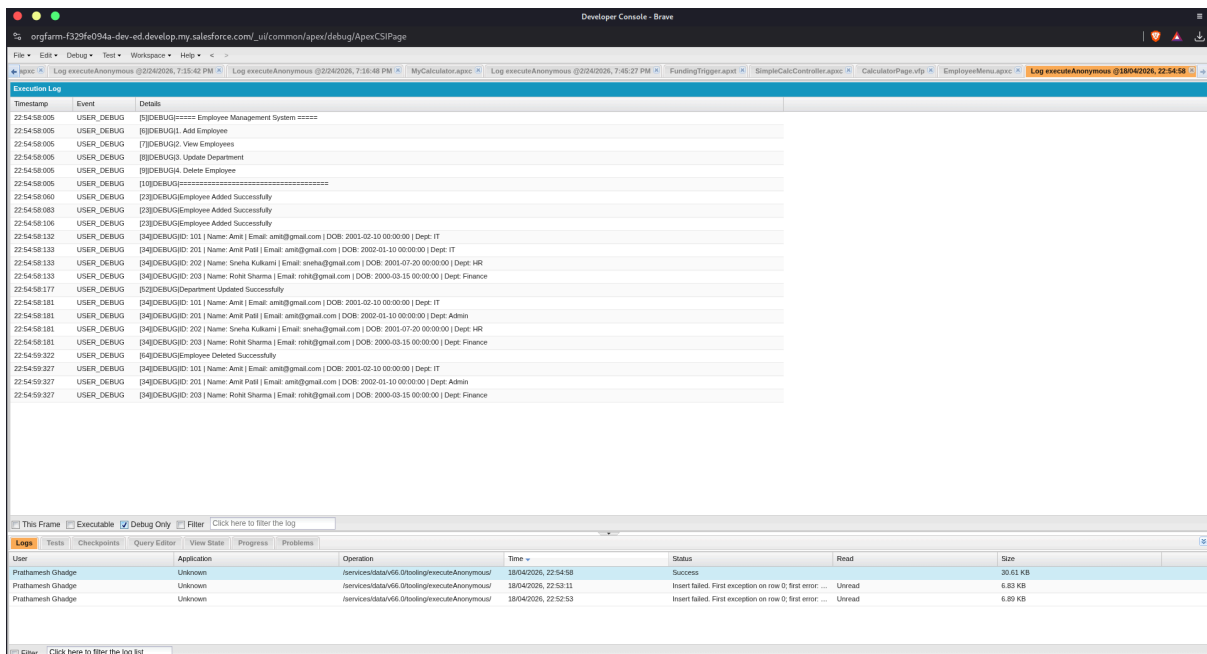
Call the Method:

Execute: Click the **Execute** button at the bottom right.



5. Viewing the Output

- **Logs Tab:** A new "Log" entry will appear in the bottom pane. Double-click it.
- **Filter Results:** In the new window, check the box that says **Debug Only**.



Creating a Calculator Application in Salesforce using Apex and Visualforce

Part 1: Create Apex Class (Logic)

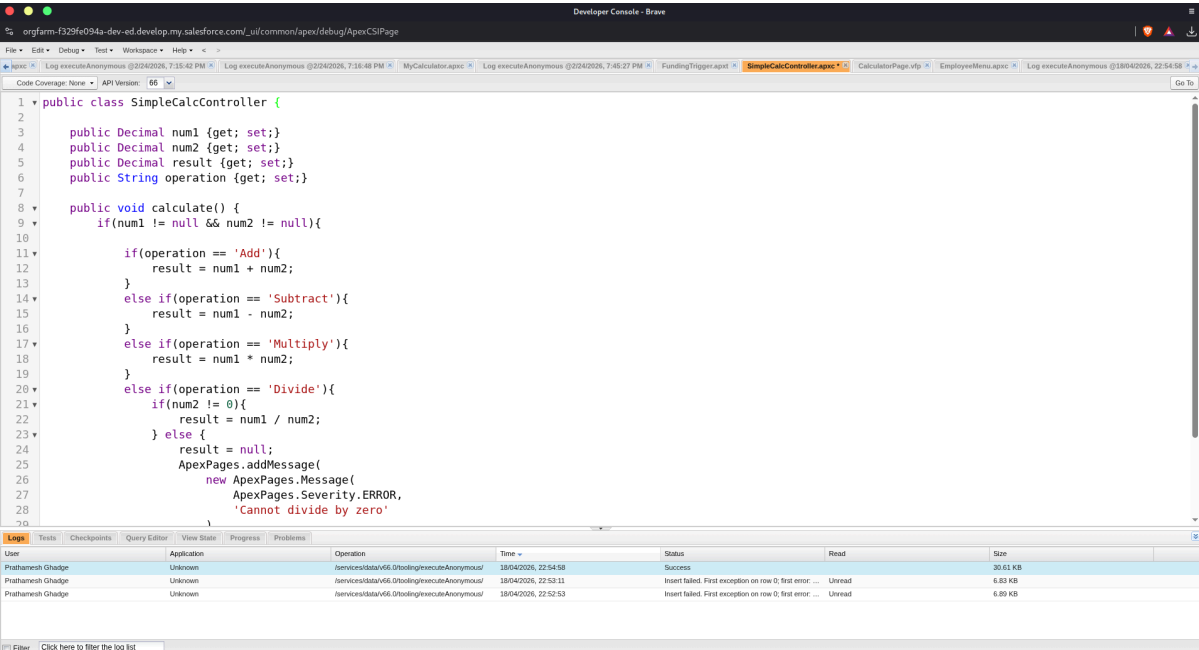
Step 1: Open Developer Console

- Click **Settings** → **Developer Console**

Step 2: Create Apex Class

- Go to **File** → **New** → **Apex Class**
- Enter name: **SimpleCalcController**
- Click **OK**

Step 3: Write Apex Code



```
1 public class SimpleCalcController {
2
3     public Decimal num1 {get; set;}
4     public Decimal num2 {get; set;}
5     public Decimal result {get; set;}
6     public String operation {get; set;}
7
8     public void calculate() {
9         if(num1 != null && num2 != null){
10
11             if(operation == 'Add'){
12                 result = num1 + num2;
13             }
14             else if(operation == 'Subtract'){
15                 result = num1 - num2;
16             }
17             else if(operation == 'Multiply'){
18                 result = num1 * num2;
19             }
20             else if(operation == 'Divide'){
21                 if(num2 != 0){
22                     result = num1 / num2;
23                 } else {
24                     result = null;
25                     ApexPages.addMessage(
26                         new ApexPages.Message(
27                             ApexPages.Severity.ERROR,
28                             'Cannot divide by zero'
29                         )
30                     );
31                 }
32             }
33         }
34     }
35 }
```

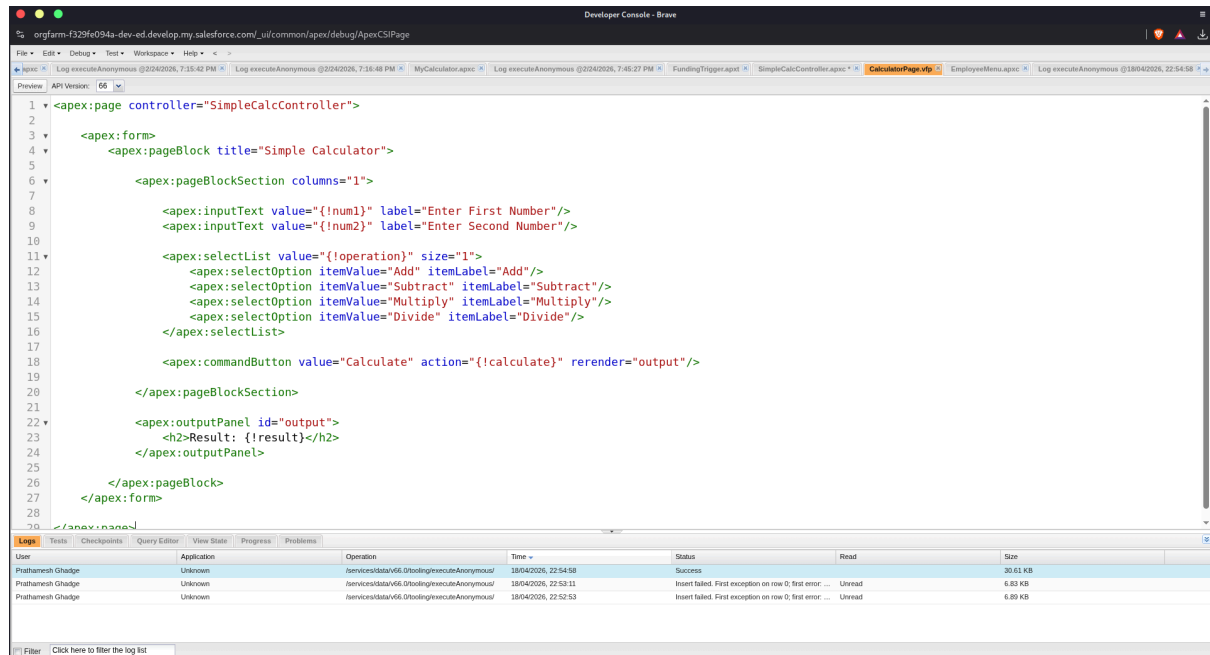
User	Application	Operation	Time	Status	Read	Size
Prathamesh Ghadge	Unknown	/services/data/v66.0/tooling/executeAnonymous/	18/04/2026, 22:54:58	Success		30.61 KB
Prathamesh Ghadge	Unknown	/services/data/v66.0/tooling/executeAnonymous/	18/04/2026, 22:53:11	Insert failed. First exception on row 0: first error: ...	Unread	6.83 KB
Prathamesh Ghadge	Unknown	/services/data/v66.0/tooling/executeAnonymous/	18/04/2026, 22:52:53	Insert failed. First exception on row 0: first error: ...	Unread	6.89 KB

Part 2: Create Visualforce Page (UI)

Step 4: Create Visualforce Page

- Go to **File** → **New** → **Visualforce Page**
- Enter name: **CalculatorPage**

Step 5: Write Visualforce Code



```
1 <apex:page controller="SimpleCalcController">
2
3   <apex:form>
4     <apex:pageBlock title="Simple Calculator">
5
6       <apex:pageBlockSection columns="1">
7
8         <apex:inputText value="{!num1}" label="Enter First Number"/>
9         <apex:inputText value="{!num2}" label="Enter Second Number"/>
10
11        <apex:selectList value="{!operation}" size="1">
12          <apex:selectOption itemValue="Add" itemLabel="Add"/>
13          <apex:selectOption itemValue="Subtract" itemLabel="Subtract"/>
14          <apex:selectOption itemValue="Multiply" itemLabel="Multiply"/>
15          <apex:selectOption itemValue="Divide" itemLabel="Divide"/>
16        </apex:selectList>
17
18        <apex:commandButton value="Calculate" action="{!calculate}" reRender="output"/>
19
20      </apex:pageBlockSection>
21
22      <apex:outputPanel id="output">
23        <h2>Result: {!result}</h2>
24      </apex:outputPanel>
25
26    </apex:pageBlock>
27  </apex:form>
28</apex:page>
```

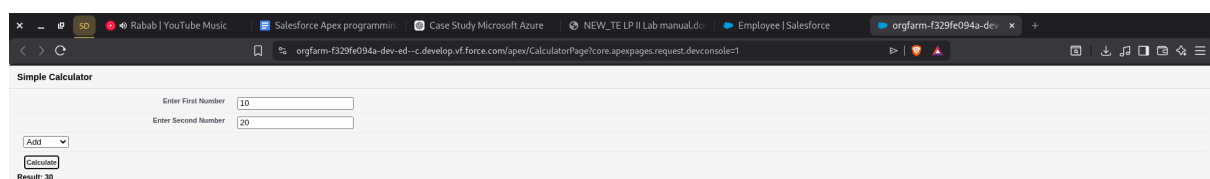
Part 3: Run the Application

Step 6: Preview Page

- Click **Preview** button

Step 7: Use Calculator

- Enter two numbers
- Select operation
- Click **Calculate**
- View result



Simple Calculator

Enter First Number: 10

Enter Second Number: 20

Operation: Add

Calculate

Result: 30

9. Conclusion

This assignment helped in understanding how to build applications in Salesforce using Apex and Visualforce. It demonstrated how backend logic and frontend UI work together.

Assignment No: 10

1. Title

Design and Development of Custom Application (Mini Project) using Salesforce Cloud

2. Problem Statement

Salesforce provides a platform to design and develop custom applications using Apex, Visualforce, and custom objects.

The objective of this mini project is to design and implement custom applications to automate real-world processes using Salesforce Cloud.

3. Objectives

- To design custom applications in Salesforce
- To create and manage custom objects and fields
- To implement business logic using Apex
- To automate processes using triggers
- To build user-friendly interfaces

4. Working of System (Short Explanation)

1. User enters candidate details
2. Trigger calls Apex class
3. Score is calculated automatically
4. Data is stored in object
5. Visualforce page displays ranked candidates

5. Applications

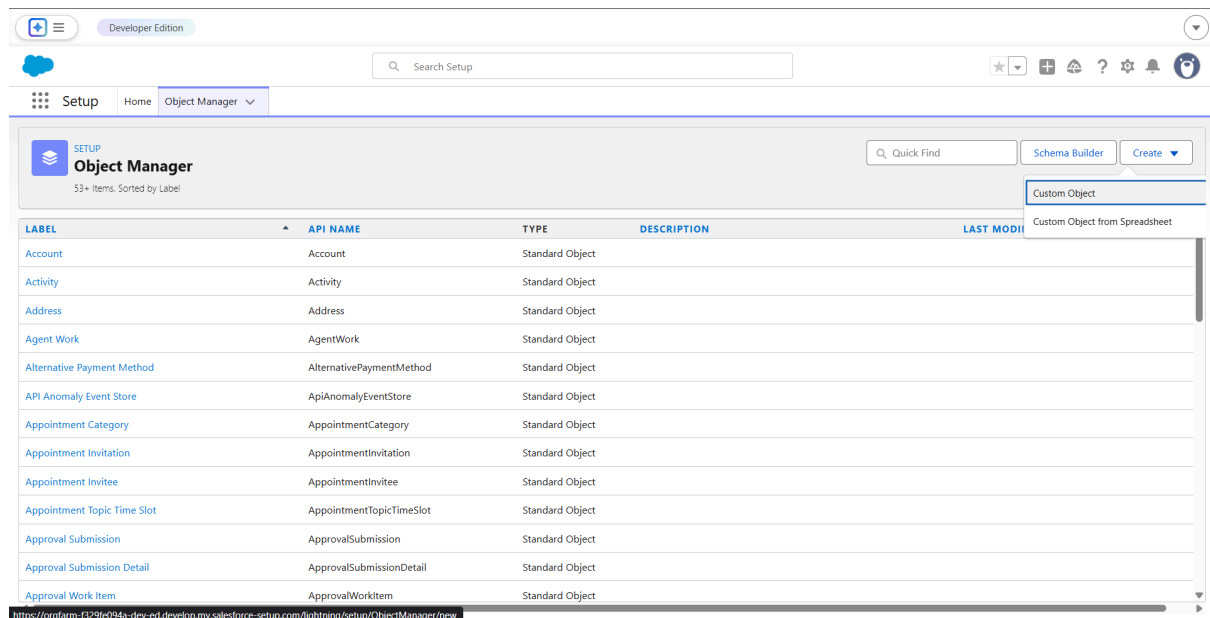
- Recruitment systems
- HR management
- Candidate filtering
- Job portals

AI-Based Recruitment Management System

Phase 1: Database Setup (Custom Object)

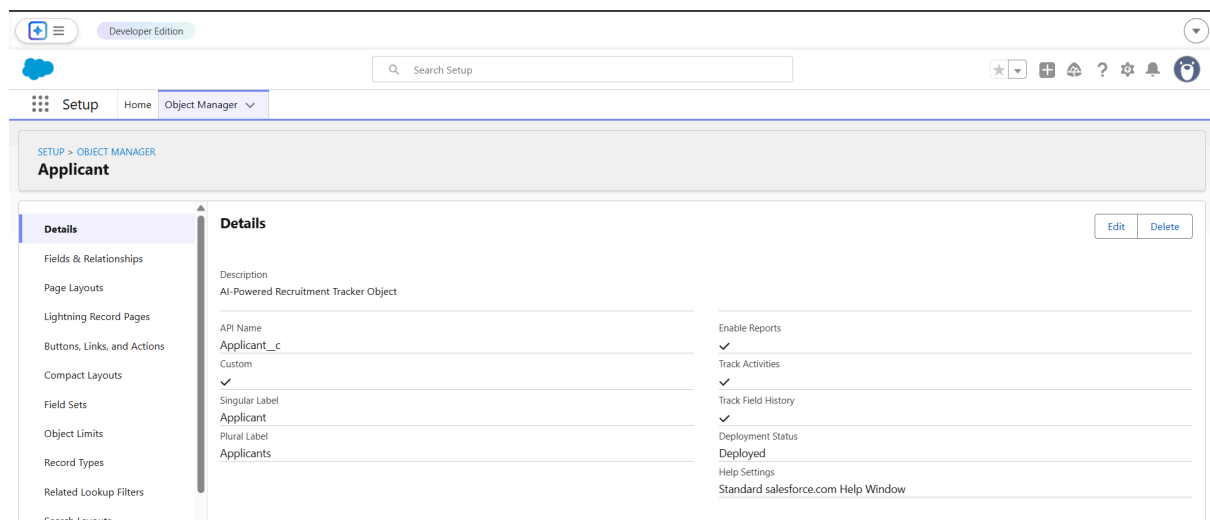
Step: Create Applicant Object

Go to Setup → Object Manager → Create → Custom Object → Enter Label: Applicant, Plural: Applicants, Record Name: Applicant Name (Text) → ✓ Enable Allow Reports, Launch Tab Wizard → Click Save



The screenshot shows the Salesforce Object Manager interface. The top navigation bar includes the Setup menu, Home link, and Object Manager dropdown. The main header area displays the Object Manager title, a search bar, and buttons for Schema Builder and Create. A dropdown menu is open, showing 'Custom Object' and 'Custom Object from Spreadsheet'. Below this is a table listing standard objects.

LABEL	API NAME	TYPE	DESCRIPTION	LAST MODIFIED
Account	Account	Standard Object		
Activity	Activity	Standard Object		
Address	Address	Standard Object		
Agent Work	AgentWork	Standard Object		
Alternative Payment Method	AlternativePaymentMethod	Standard Object		
API Anomaly Event Store	ApiAnomalyEventStore	Standard Object		
Appointment Category	AppointmentCategory	Standard Object		
Appointment Invitation	AppointmentInvitation	Standard Object		
Appointment Invitee	AppointmentInvitee	Standard Object		
Appointment Topic Time Slot	AppointmentTopicTimeSlot	Standard Object		
Approval Submission	ApprovalSubmission	Standard Object		
Approval Submission Detail	ApprovalSubmissionDetail	Standard Object		
Approval Work Item	ApprovalWorkItem	Standard Object		



The screenshot shows the Salesforce Object Manager interface for the 'Applicant' custom object. The left sidebar contains a list of configuration options: Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, and Search Layouts. The main area displays the 'Details' tab for the 'Applicant' object.

Details

Description: AI-Powered Recruitment Tracker Object

API Name: Applicant__c

Custom: ☒

Singular Label: Applicant

Plural Label: Applicants

Enable Reports: ☒

Track Activities: ☒

Track Field History: ☒

Deployment Status: Deployed

Help Settings: Standard salesforce.com Help Window

Step: Create Custom Fields

Email: (Data Type: **Email**) | Label: Email

Experience: (Data Type: **Number**, 2, 0) | Label: Experience Years

Technical Skills: (Data Type: **Long Text Area**) | Label: Technical Skills

Priority Score: (Data Type: **Number**, 3, 0) | Label: Priority Score

Developer Edition

Search Setup

Setup Home Object Manager

SETUP > OBJECT MANAGER

Applicant

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Restriction Rules

Scoping Rules

Fields & Relationships

8 Items, Sorted by Field Label

Quick Find

New Deleted Fields Field Dependencies Set History Tracking

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Applicant Name	Name	Text(80)		✓
Created By	CreatedById	Lookup(User)		
Email	Email__c	Email		
Experience Years	Experience_Years__c	Number(2, 0)		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Priority Score	Priority_Score__c	Number(3, 0)		
Technical Skills	Technical_Skills__c	Long Text Area(32768)		

Phase 2: Backend Logic (Apex Class)

Step: Create Apex Class

1. Go to **Developer Console** → **File** → **New** → **Apex Class**
Name: **ApplicantService**

Developer Console - Brave

orgfarm-f329fe094a-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage

File Edit Debug Test Workspace Help

2/24/2026, 7:15:42 PM Log executeAnonymous @2/24/2026, 7:16:48 PM MyCalculator.apex Log executeAnonymous @2/24/2026, 7:45:27 PM FundingTrigger.apxt SimpleCalcController.apex CalculatorPage.vfp ApplicantService.apex

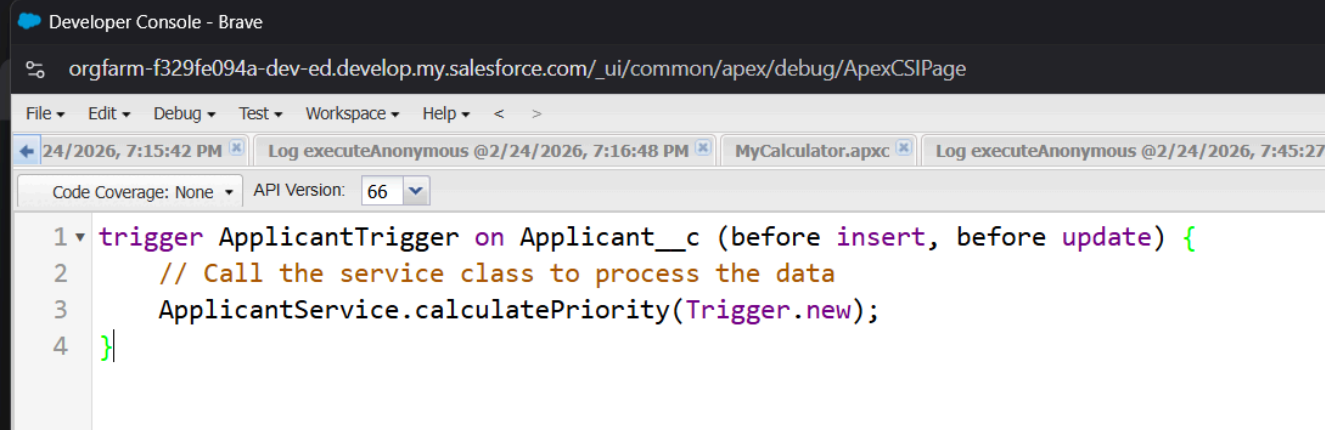
Code Coverage: None API Version: 66

```
1 public class ApplicantService {
2     public static void calculatePriority(List<Applicant__c> applicants) {
3         for (Applicant__c app : applicants) {
4             Integer score = 0;
5
6             // Logic 1: Scoring based on experience
7             if (app.Experience_Years__c != null) {
8                 if (app.Experience_Years__c >= 5) score += 50;
9                 else if (app.Experience_Years__c >= 2) score += 30;
10                else score += 10;
11            }
12
13            // Logic 2: Keyword matching for 'Apex' or 'Java'
14            if (app.Technical_Skills__c != null) {
15                String skills = app.Technical_Skills__c.toLowerCase();
16                if (skills.contains('apex')) score += 20;
17                if (skills.contains('java')) score += 20;
18            }
19            app.Priority_Score__c = score;
20        }
21    }
22 }
```

Phase 3: Automation (Trigger)

Step: Create Apex Trigger

Go to **Developer Console** → **File** → **New** → **Apex Trigger**
Name: **ApplicantTrigger**



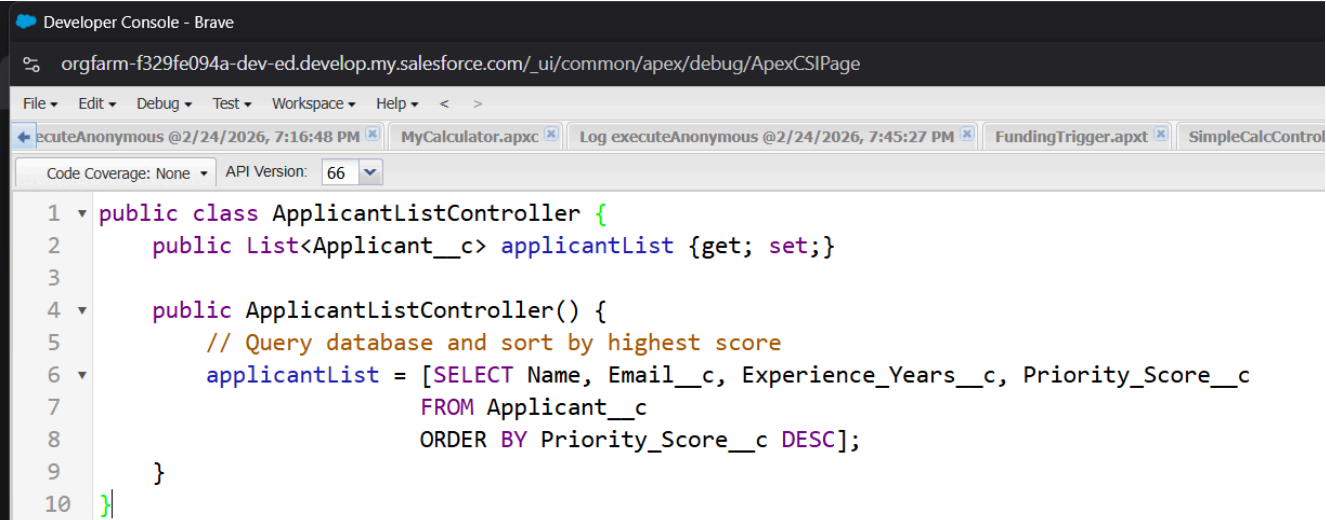
The screenshot shows the Salesforce Developer Console interface. The top bar indicates the org is 'orgfarm-f329fe094a-dev-ed.develop.my.salesforce.com'. The browser tabs show 'Log executeAnonymous @2/24/2026, 7:16:48 PM', 'MyCalculator.apxc', and 'Log executeAnonymous @2/24/2026, 7:45:27'. The code editor displays the following Apex Trigger code:

```
1 trigger ApplicantTrigger on Applicant__c (before insert, before update) {  
2     // Call the service class to process the data  
3     ApplicantService.calculatePriority(trigger.new);  
4 }
```

Phase 4: Custom UI (Leaderboard Controller)

Step: Create Apex Controller

Go to **File** → **New** → **Apex Class**
Name: **ApplicantListController**

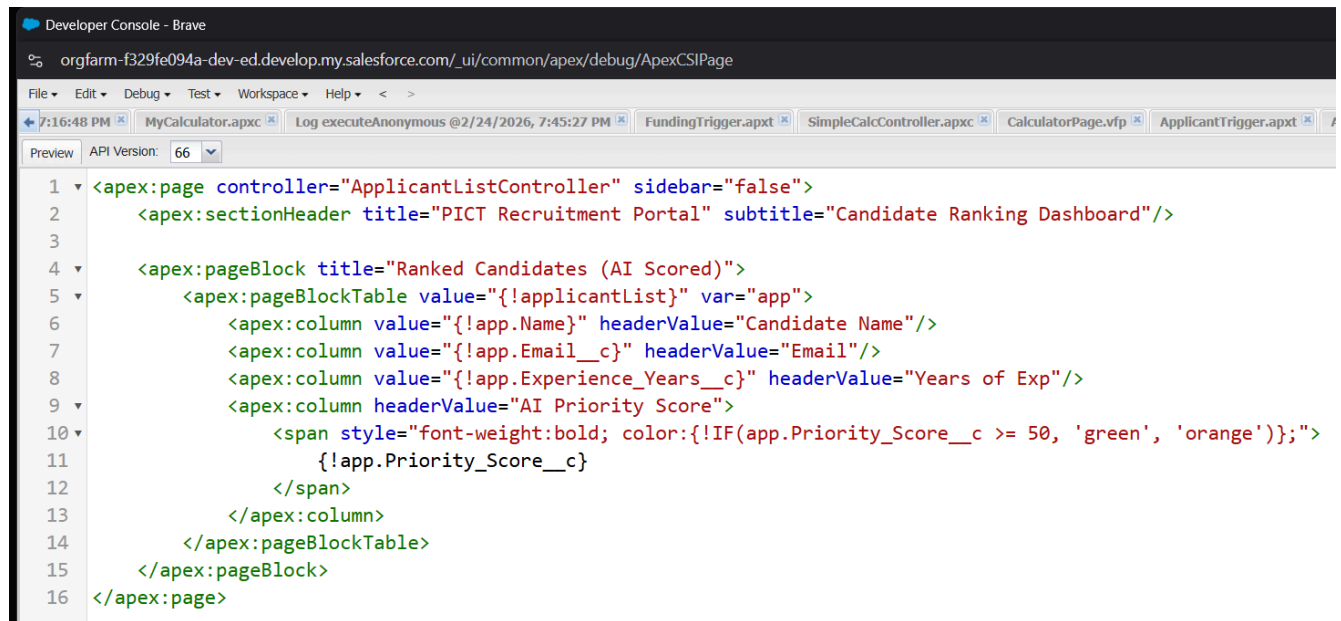


The screenshot shows the Salesforce Developer Console interface. The top bar indicates the org is 'orgfarm-f329fe094a-dev-ed.develop.my.salesforce.com'. The browser tabs show 'Log executeAnonymous @2/24/2026, 7:16:48 PM', 'MyCalculator.apxc', 'Log executeAnonymous @2/24/2026, 7:45:27 PM', 'FundingTrigger.apxt', and 'SimpleCalcControl'. The code editor displays the following Apex Class code:

```
1 public class ApplicantListController {  
2     public List<Applicant__c> applicantList {get; set;}  
3  
4     public ApplicantListController() {  
5         // Query database and sort by highest score  
6         applicantList = [SELECT Name, Email__c, Experience_Years__c, Priority_Score__c  
7                             FROM Applicant__c  
8                             ORDER BY Priority_Score__c DESC];  
9     }  
10 }
```

Phase 5: Visualforce Page

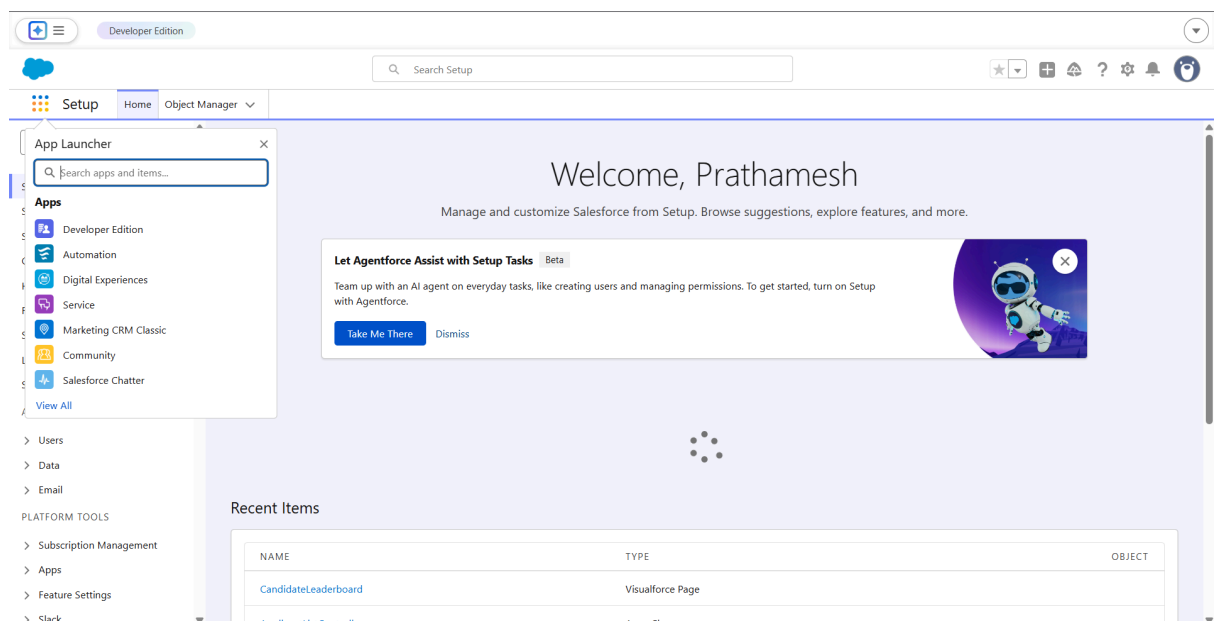
1. File > New > Visualforce Page. Name: CandidateLeaderboard.
2. Save (Ctrl+S)

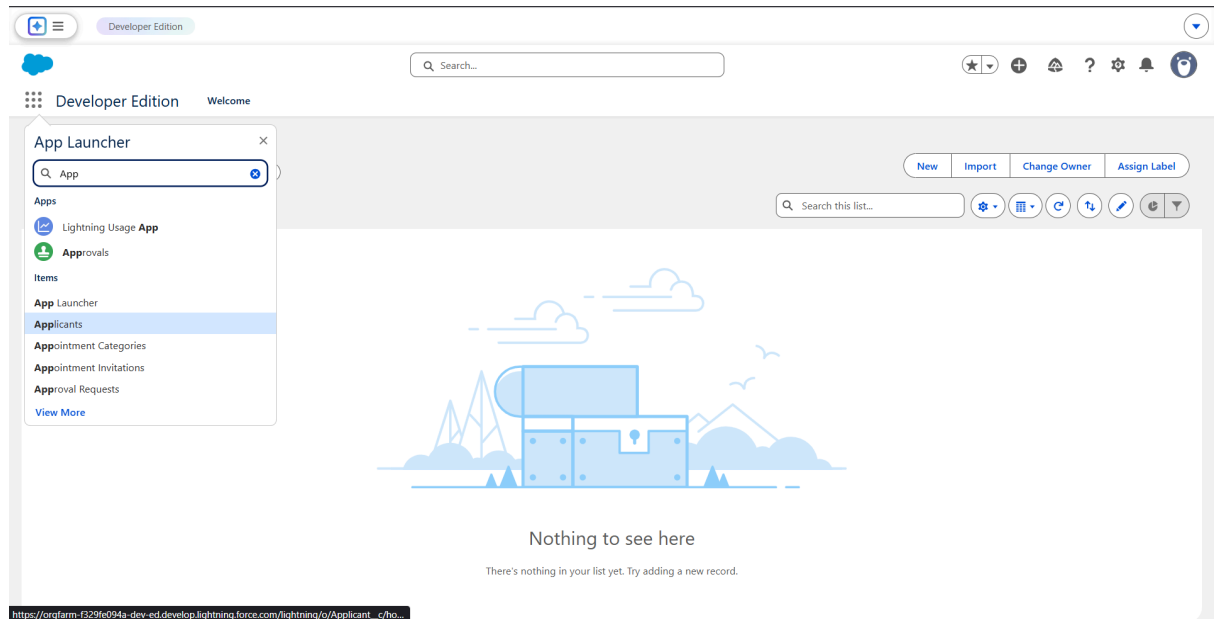


```
1 <apex:page controller="ApplicantListController" sidebar="false">
2   <apex:sectionHeader title="PICT Recruitment Portal" subtitle="Candidate Ranking Dashboard"/>
3
4   <apex:pageBlock title="Ranked Candidates (AI Scored)">
5     <apex:pageBlockTable value="{!applicantList}" var="app">
6       <apex:column value="{!app.Name}" headerValue="Candidate Name"/>
7       <apex:column value="{!app.Email__c}" headerValue="Email"/>
8       <apex:column value="{!app.Experience_Years__c}" headerValue="Years of Exp"/>
9       <apex:column headerValue="AI Priority Score">
10        <span style="font-weight:bold; color:{!IF(app.Priority_Score__c >= 50, 'green', 'orange')};">
11          {!app.Priority_Score__c}
12        </span>
13      </apex:column>
14    </apex:pageBlockTable>
15  </apex:pageBlock>
16</apex:page>
```

Phase 6: Implementation & Testing

1. Input Data: Go to the App Launcher (9 dots) → Applicants → Click New.





2. Create 3 records:

- *Candidate A*: 6 years Exp, Skills: "Apex, Java" (Expected Score: 90)
- *Candidate B*: 3 years Exp, Skills: "Python" (Expected Score: 30)
- *Candidate C*: 1 year Exp, Skills: "Java" (Expected Score: 30)

The screenshot shows the 'New Applicant' form in Salesforce Developer Edition. The form is titled 'New Applicant' and contains the following fields:

- Applicant Name**: Prathamesh
- Email**: prathamtech24@gmail.com
- Experience Years**: 6
- Technical Skills**: Apex, Java
- Priority Score**: (empty)

The Owner is set to Prathamesh Ghadge. The form also includes a 'Cancel' button, a 'Save & New' button, and a 'Save' button.

Developer Edition

Search...

★

+

?

⚙

🔔

Developer Edition

Welcome

Applicants

Recently Viewed ▾

New

Import

Change Owner

Assign Label

Search this list...

⚙

☐ Applicant Name

▾

1

☐ Pranav

▾

2

☐ Heena

▾

3

☐ Prathamesh

▾

3. **View Results:** Go back to your **Visualforce Page** in the Developer Console and click **Preview**.

PICT Recruitment Portal

Candidate Ranking Dashboard

Ranked Candidates (AI Scored)			
Candidate Name	Email	Years of Exp	AI Priority Score
Prathamesh	prathamtech24@gmail.com	6	90
Heena	heena14@gmail.com	2	50
Pranav	pranav29@gmail.com	3	30